Information Retrieval

Lab 2: ***Simple Index***

2020/2021

Simple Index

Index is a tool for finding records in a file.

It consists of a *key field* on which the index is searched and a *reference (address or Byte Offset) field* that tells where to find the data file record associated with a particular key.

Example : The index to a book (usually at the start of the book) provides a *way to find* a topic quickly. Imagine a book without an index?

Key	Byte-Offset

Simple Index

Index files are typically much smaller than the original file

Records on Simple Index file are sorted by the key, unlike the data file which usually have the data unsorted.

Simple Index

Index

Key	Reference field
ANG3795	152
COL31809	338
COL38358	196
DG139201	382
DG18807	241
FF245	427
LON2312	17
MER75016	285
RCA2626	62
WAR23699	117

Data File

Record address	ID Label number	Title	Composer(s)	Artist(s)
17	LON 2312	Romeo and Juliet	Prokofiev	Maazel
62	RCA 2626	Quartet in C Sharp Minor	Beethoven	Julliard
117	WAR 23699	Touchstone	Corea	Corea
152	ANG 3795	Symphony No. 9	Beethoven	Giulini
196	COL 38358	Nebraska	Springsteen	Springsteen
241	DG 18807	Symphony No. 9	Beethoven	Karajan
285	MER 75016	Coq d'Or Suite	Rimsky-Korsakov	Leinsdorf
338	COL 31809	Symphony No. 9	Dvorak	Bernstein
382	DG 139201	Violin Concerto	Beethoven	Ferras
427	FF 245	Good News	Sweet Honey in the Rock	Sweet Honey in the Rock

Figure 7.2 Contents of sample recording file.

Simple Index

Simple index is kept on memory when the index is small enough.

If the index is to large to be kept on memory, it has to be kept on disk as index file just like the data file.

Our main assumption, **that index can’t be hold on memory.**

Operations on Data File : Record Addition

Adding a record to the data file implies adding a record to the index file.

Adding a record to the data file won’t be a problem as long as it’s not sorted, just append the record to the end of the file.

As for the index file which is kept sorted, adding a new record would require rearranging the records in this index to keep the order. (shifting the records to add the new record)

Operations on Data File : Record Deletion

Deleting a data record from the data file requires deletion of the corresponding index record from the index file.

There are two methods to delete a record from a file:

- Search for the record on the file and put a mark on it that it’s deleted
- Search for the record on the file and shift all records beneath it one position to the up.

Operations on Data File : Record Update

There are two categories of updates:

- the update modifies the value of the key
- the update does not modify the value of the key

If the update modifies the key, then re-ordering/re-sort of the index file might be required.

- If the update does not modify the value of the key but modify other attributes of the record, then it may change all the bytes offsets/addresses of all records on the file. An update for the index file would be required

Operations on Data File : Search

To search a record on data file, you can use the sorted index file.

The best way to search a sorted list is binary search.

Binary search for the index file would cost more than binary search for an array due to the seek time required to access the file and read the integers.

Why Simple Index isn’t a Good Solution !

- Index rearrangement (at addition/updating/deletion) of records be very time consuming.

- Searching the file with binary search would cost more time due to the number of seeks we have to do to reach the record.

Simple Index: Pseudo Code for Addition

Adding a Record

```
DataFile.seek(Datafile.length)
DataFile.Write(NewRecord)

IndexFile.seek(0);
While(IndexFile Not EOF)
{
        CurrentRecord = IndexFile.ReadRecord()
        If(CurrentRecord.Key >NewRecord.Key)
        {
                IndexFile.seek (- RecordSize)
                Indexfile.write (NewRecord)
                NewRecord = CurrentRecord;

        }
}
IndexFile.write (NewRecord)
```